Database Design: Entity Relationship Model

for

MomentsHub: A Collaborative Daily Photo Capsule

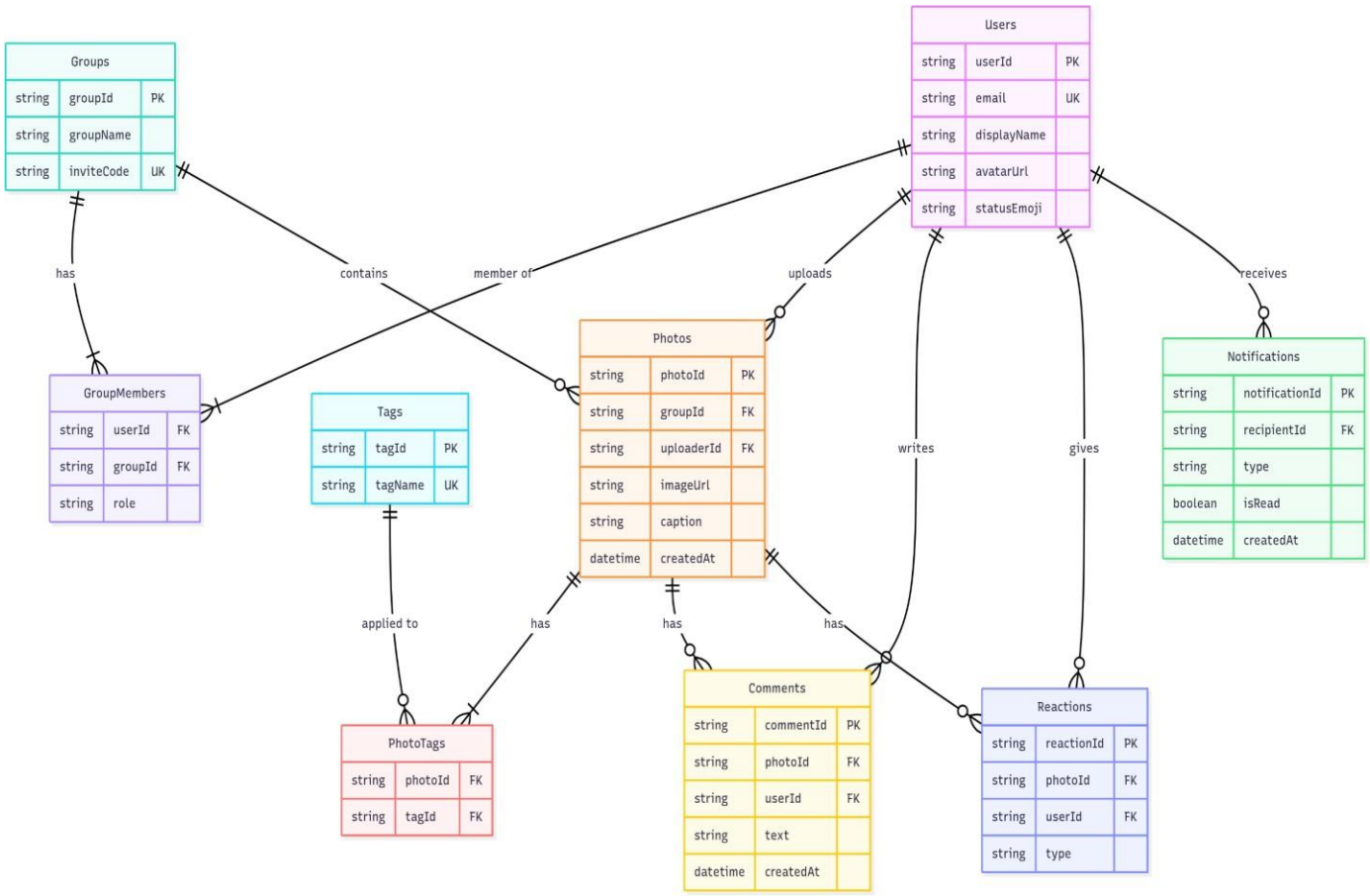
Submitted by:

Name: Sasmith G
Reg. No: 23BPS1074

1. Introduction

This document provides a detailed description of the database schema for the **MomentsHub** project, a collaborative photo sharing web application. The database is designed to support all core functionalities, including user management, private groups, photo sharing, and rich social interactions. The following Entity Relationship (ER) Diagram provides a visual representation of the data model, which is followed by a comprehensive breakdown of its components.

2. Entity-Relationship (ER) Diagram



3. Entity Descriptions

The database is composed of **strong entities**, which represent the core objects of the system, and **associative entities** (also known as junction tables), which are used to model many-to-many relationships between strong entities.

3.1 Strong Entities

These entities can exist independently and form the backbone of the application.

- **Users**
  - **Description:** This entity stores all information related to a registered user's account.
  - **Attributes:**
    - userId (PK): A unique identifier for each user, provided by Firebase Authentication.
    - email (UK): The user's unique email address, used for login.
    - displayName: The user's public name displayed within the app.
    - avatarUrl: A URL pointing to the user's profile picture.
    - statusEmoji: A string containing a single emoji representing the user's current status.
- **Groups**
  - **Description:** This entity represents a private, shared space where a collection of users can post photos.
  - **Attributes:**
    - groupId (PK): A unique identifier for each group.
    - groupName: The custom name of the group (e.g., "Family Trip 2025").
    - inviteCode (UK): A unique, short code used to invite others to the group.
- **Photos**
  - **Description:** This entity stores the metadata for every photo uploaded to the application.
  - **Attributes:**
    - photoId (PK): A unique identifier for each photo.
    - groupId (FK): A foreign key linking the photo to the Groups entity it belongs to.
    - uploaderId (FK): A foreign key linking the photo to the Users entity who uploaded it.
    - imageUrl: A URL pointing to the actual image file stored in Firebase Storage.
    - caption: The text description accompanying the photo.

        - createdAt: A timestamp recording when the photo was uploaded, crucial for chronological sorting and the "On This Day" feature.
- **Tags**
    - **Description:** This entity stores the unique names of tags that can be applied to photos, enabling categorization and search.
    - **Attributes:**
        - tagId (PK): A unique identifier for each tag.
        - tagName (UK): The unique tag text (e.g., "beach", "birthday").
- **Notifications**
    - **Description:** This entity stores records of notifications that need to be displayed to users.
    - **Attributes:**
        - notificationId (PK): A unique identifier for each notification.
        - recipientId (FK): A foreign key linking to the Users entity who should receive the notification.
        - type: A string indicating the type of notification (e.g., "mention", "new_photo").
        - isRead: A boolean flag to track if the user has viewed the notification.
        - createdAt: A timestamp recording when the notification was generated.

3.2 Associative Entities (Junction Tables)

These entities exist to connect strong entities in many-to-many relationships.

- **GroupMembers**
    - **Description:** This entity links users to groups, allowing a user to be a member of many groups and a group to have many members.
    - **Attributes:**
        - userId (FK): A foreign key referencing the Users entity.
        - groupId (FK): A foreign key referencing the Groups entity.
        - role: A string defining the user's role in the group (e.g., "admin", "member").
- **PhotoTags**
    - **Description:** This entity links photos to tags, allowing a photo to have multiple tags and a tag to be used on multiple photos.
    - **Attributes:**
        - photoId (FK): A foreign key referencing the Photos entity.
        - tagId (FK): A foreign key referencing the Tags entity.
- **Comments**

    - **Description:** This entity stores every comment made on a photo, linking the comment text to a specific user and photo.
    - **Attributes:**
        - commentId (PK): A unique identifier for each comment.
        - photoId (FK): A foreign key referencing the Photos entity.
        - userId (FK): A foreign key referencing the Users entity who wrote the comment.
        - text: The content of the comment.
        - createdAt: A timestamp recording when the comment was posted.
- **Reactions**
    - **Description:** This entity stores every emoji reaction made on a photo, linking the reaction to a specific user and photo.
    - **Attributes:**
        - reactionId (PK): A unique identifier for each reaction.
        - photoId (FK): A foreign key referencing the Photos entity.
        - userId (FK): A foreign key referencing the Users entity who gave the reaction.
        - type: A string representing the emoji reaction (e.g., "♡", "😂").

4. Relationship Descriptions

The relationships define how entities interact with one another.

- **Users and Groups (Many-to-Many)**
    - **Description:** A user can be a member of many groups, and a group can have many members. This relationship is modeled through the GroupMembers associative entity.
    - **Cardinality:** Users to GroupMembers is **One-to-Many**. Groups to GroupMembers is **One-to-Many**.
- **Users and Photos (One-to-Many)**
    - **Description:** A single user can upload many photos, but each photo is uploaded by only one user.
    - **Cardinality:** (1,1) from Photos to Users and (0,N) from Users to Photos.
- **Groups and Photos (One-to-Many)**
    - **Description:** A group can contain many photos, but each photo belongs to only one group.
    - **Cardinality:** (1,1) from Photos to Groups and (0,N) from Groups to Photos.
- **Photos and Tags (Many-to-Many)**
    - **Description:** A photo can have multiple tags, and a single tag can be applied

to many different photos. This is modeled through the PhotoTags associative entity.
    - **Cardinality:** Photos to PhotoTags is **One-to-Many**. Tags to PhotoTags is **One-to-Many**.
- **Photos, Users, Comments, and Reactions (One-to-Many)**
    - **Description:** These relationships model the social interactions. A photo can have many comments and many reactions. A user can write many comments and give many reactions. Each comment and reaction, however, belongs to only one photo and is made by only one user.
    - **Cardinality:** These are all modeled as **One-to-Many** relationships from the parent entities (Users, Photos) to the child entities (Comments, Reactions).